JDK vs JRE vs JVM - Zero to Interview Guide

Easy language, deep concepts, hands-on practice, exam and interview answers

BEGINNER FRIENDLY	HANDS-ON LABS	INTERVIEW READY

Recommended practice environment: JDK 25 LTS or a newer compatible JDK

Learning roadmap

This guide explains the three most confused Java terms using a simple analogy, a modern technical view and command-line experiments.

- **JVM:** The engine that loads and executes Java bytecode.
- **Runtime environment:** The JVM plus libraries and supporting runtime components needed by the application.
- **JDK:** The complete development kit with the runtime and developer tools.
- **Modern truth:** JDK 11 and later distributions do not contain the old separate jre/ directory image used by older JDKs.
- **Practice:** Check tools, compile code, inspect runtime properties and create a custom runtime image.

WHY THIS TOPIC MATTERS: Many exam answers use the old formula JDK = JRE + development tools. The idea is useful, but modern JDK layouts changed after Java 9, and JDK 11+ no longer ships the old separate JRE image inside the JDK. A strong answer explains both the concept and the modern reality.

Contents

1. Kitchen analogy

2. Big-picture diagram

3. JVM in depth

4. Runtime environment in depth

5. JDK tools in depth

6. Compile and run cycle

7. Hands-on command lab

8. Inspect the current runtime

9. Build a custom runtime with jlink

10. PATH and JAVA_HOME

11. Comparison table

12. Exam and interview answers

13. Practice and revision

1. Kitchen analogy

Java term	Kitchen analogy	Meaning
JVM	The cook	Executes the recipe instructions.
Runtime environment	Cook + kitchen + basic ingredients	Everything needed to serve the prepared dish.
JDK	Full cooking school and workshop	Runtime plus tools for creating, testing, packaging and documenting recipes.

SIMPLE MEMORY LINE: JVM runs bytecode. The runtime supplies what the program needs while running. The JDK gives developers tools to build and run Java software.

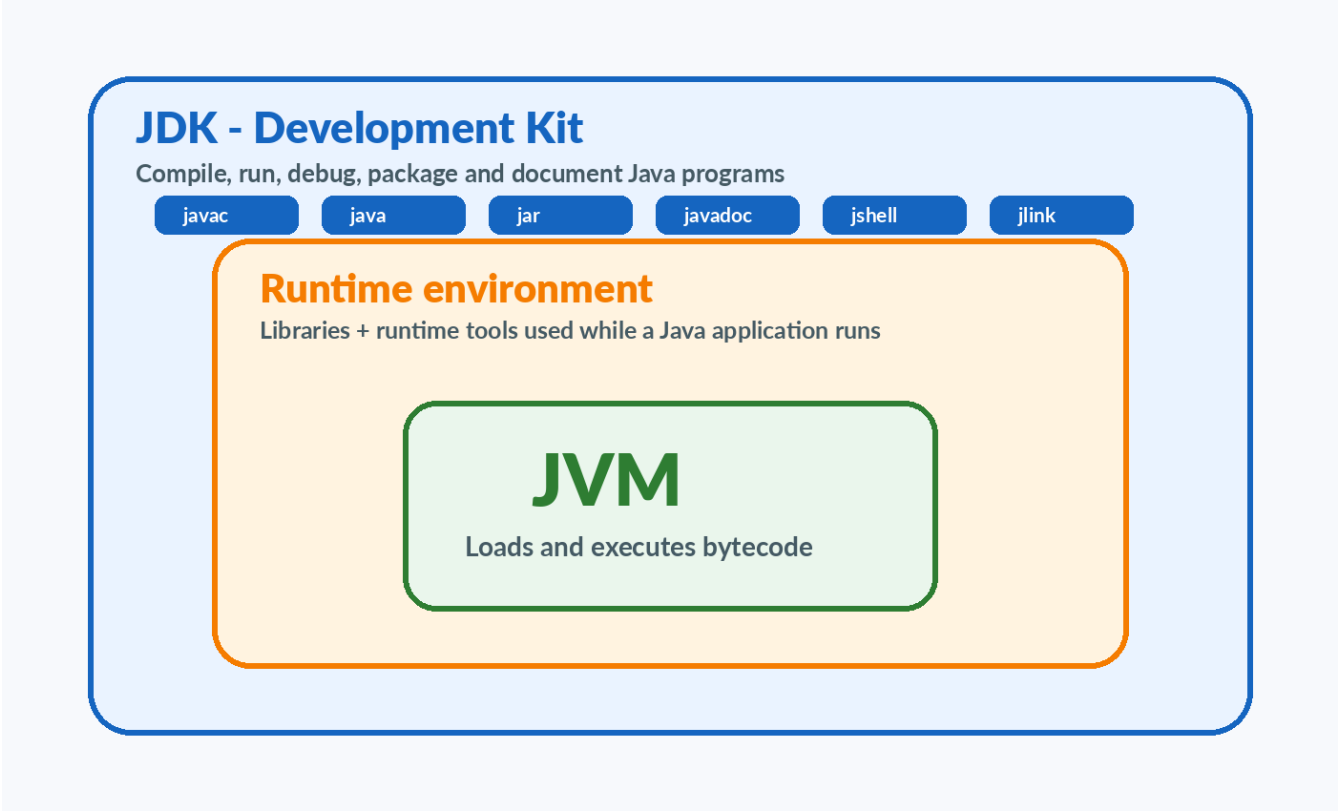
TECHNICAL DEFINITION

The JVM is a virtual execution machine defined by a specification; a Java runtime provides the JVM and core runtime libraries; the JDK provides the runtime plus development and diagnostic tools.

IN SIMPLE WORDS

JVM = engine, runtime = engine plus running support, JDK = full developer toolbox.

2. Big-picture relationship



MODERN LAYOUT NOTE: The nested picture is a conceptual model. Starting with JDK 11, the installed JDK no longer contains the old separate jre/ image. Developers normally install a JDK or package a purpose-built runtime.

Need	What you normally use
Write and compile Java code	JDK
Run code during development	JDK - it includes the launcher and runtime components
Ship a desktop or server application	A JDK-based runtime, container image, vendor runtime, or custom jlink image
Execute bytecode internally	JVM implementation

3. JVM in depth

- **Class loader subsystem:** Finds and loads class definitions.
- **Bytecode verification:** Checks class-file structure and important safety rules.
- **Runtime data areas:** Manage stacks, heap, method metadata and other execution state.
- **Execution engine:** Interprets bytecode and uses JIT compilers for hot methods.
- **Garbage collector:** Reclaims memory for unreachable managed objects.
- **Native interface:** Allows controlled interaction with native code when required.
- **Thread management:** Maps Java execution to operating-system and runtime scheduling mechanisms.

SPECIFICATION VS IMPLEMENTATION: The JVM specification defines required behavior. HotSpot, OpenJ9 and other JVMs are implementations. They can use different internal strategies while running valid class files correctly.

Ask the running JVM to identify itself

```
public class VmInfo {
    public static void main(String[] args) {
        System.out.println(System.getProperty("java.vm.name"));
        System.out.println(System.getProperty("java.vm.vendor"));
        System.out.println(System.getProperty("java.vm.version"));
    }
}
```

4. Runtime environment in depth

- **JVM:** The execution engine.
- **Core modules and libraries:** Types such as String, List, Files, HTTP client and many others.
- **Launcher and support files:** The java command starts an application and configures the runtime.
- **Configuration and security material:** Certificates, providers, locale data and runtime configuration.
- **Platform integration:** Native components that connect Java APIs to the local OS and CPU.

HISTORICAL JRE: Before JDK 9, users commonly downloaded a separate JRE, and a JDK contained a jre/ directory. JDK 9 and 10 used modular runtime images. JDK 11 and later do not include the old separate JRE image in the installed JDK.

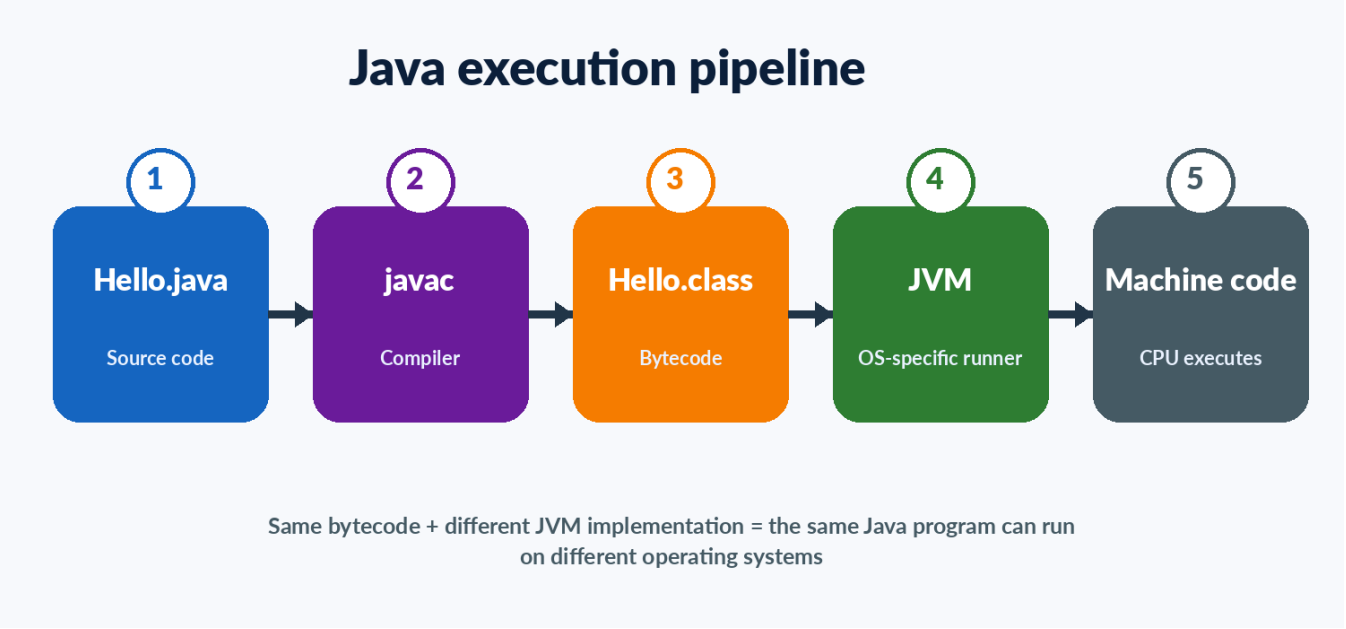
WHAT SHOULD A BEGINNER INSTALL?: Install a current JDK. JDK 25 is the current LTS release, while JDK 26 is the current feature release as of July 2026. A JDK includes the tools you need for learning and also runs your programs.

5. JDK in depth - the developer toolbox

Tool	Purpose	Beginner command
java	Launch a class, JAR or source-file program	java Main
javac	Compile .java source files into .class files	javac Main.java
jshell	Interactive Java shell for quick experiments	jshell
jar	Create, inspect and update JAR archives	jar --create --file app.jar Main.class
javadoc	Generate API documentation from source comments	javadoc Main.java
javap	Inspect class-file information and bytecode	javap -c Main
jlink	Build a custom modular runtime image	jlink --add-modules java.base ...
jpackage	Package an application for a target OS	jpackage --name MyApp ...
jdb	Command-line debugger	jdb Main
jcmd	Send diagnostic commands to a running JVM	jcmd -l

EXAM ANSWER: JDK is a software development kit that includes the Java compiler, launcher, standard APIs and tools for building, running, debugging, documenting and packaging Java applications.

6. Compile and run cycle



- **JDK tool javac:** Reads Main.java and produces Main.class.
- **Runtime launcher java:** Starts a JVM, loads Main.class and invokes the main method.
- **JVM:** Executes bytecode using the local runtime implementation.
- **JDK library modules:** Supply classes used by your program, such as java.lang and java.util.

COMMON MISTAKE: javac is not part of a runtime-only environment. If java works but javac does not, you may have installed only a runtime distribution or your PATH points to the wrong folder.

7. Hands-on command lab

1

Verify your JDK installation

Goal: Prove which commands are available and which versions are active.

Run these commands in a new terminal

```
java -version
javac -version
java --list-modules
jshell --version
```

Command	What success means
java -version	The Java launcher can find a runtime and start a JVM.
javac -version	A compiler from a JDK is available.
java --list-modules	The runtime image exposes Java modules.
jshell --version	The JDK interactive shell is available.

VERSION ALIGNMENT: Ideally, java -version and javac -version should point to the intended JDK installation. Different versions can create confusing compile/run failures.

8. Hands-on compile, run and inspect

2

Use JDK tools in one small project

Goal: Compile with javac, run with java and inspect with javap.

ToolDemo.java

```
public class ToolDemo {
    public static void main(String[] args) {
        System.out.println("Compiler and runtime are working");
    }
}
```

Commands

```
javac ToolDemo.java
java ToolDemo
javap -c ToolDemo
jar --create --file tool-demo.jar ToolDemo.class
jar --list --file tool-demo.jar
```

WHAT HAPPENED: Five JDK tools worked together: javac compiled, java launched, the JVM executed, javap inspected, and jar packaged the class file.

9. Inspect the current runtime from Java code

```
public class RuntimeDetails {
    public static void main(String[] args) {
        String[] keys = {
            "java.version",
            "java.runtime.name",
            "java.runtime.version",
            "java.home",
            "java.vm.name",
            "java.vm.vendor",
            "os.name",
            "os.arch"
        };

        for (String key : keys) {
            System.out.printf("%-22s = %s%n", key,
                    System.getProperty(key));
        }
    }
}
```

- **java.home:** The home directory of the runtime used by the current process.
- **java.vm.name:** The JVM implementation name.
- **java.runtime.version:** The complete runtime version.
- **os.name and os.arch:** The local platform where this JVM is running.

USEFUL DEBUGGING HABIT: When a program behaves differently between computers, record java.version, java.home, java.vm.name, os.name and os.arch before guessing.

10. Advanced hands-on - create a custom runtime with jlink

TECHNICAL DEFINITION
jlink builds a smaller runtime image containing a chosen set of Java modules.

IN SIMPLE WORDS
Instead of asking every user to install a full JDK, you can package the parts of Java your modular application needs.

Linux/macOS style. In Windows PowerShell, place the command on one line or use PowerShell continuation syntax.

```
jlink   --add-modules java.base   --output my-runtime   --strip-debug   --no-header-files   --no-
man-pages
```

Test the generated runtime

```
my-runtime/bin/java -version
```

IMPORTANT LIMITATION: A real application may require more modules than java.base. Use jdeps and your module descriptor to determine dependencies. jlink is most natural for modular applications.

11. PATH and JAVA_HOME

TECHNICAL DEFINITION

PATH tells the command shell where to search for executable programs. JAVA_HOME is a conventional environment variable pointing to a JDK installation.

IN SIMPLE WORDS

PATH helps the terminal find java and javac. JAVA_HOME tells tools which JDK folder should be treated as home.

Setting	Example purpose	Common problem
PATH	Includes C:\Program Files\Java\jdk-25\bin or /opt/jdk-25/bin	Terminal finds an older java first.
JAVA_HOME	Points to the JDK root, not the bin directory	Points to a deleted folder or a runtime-only installation.
IDE JDK	Project SDK configured inside IntelliJ or another IDE	IDE uses a different JDK than the terminal.
Build tool JDK	Maven/Gradle process selects a Java home	Build works in IDE but fails in CI or terminal.

Windows Command Prompt

```
where java
where javac

echo %JAVA_HOME%
```

Linux or macOS shell

```
which java
which javac

echo $JAVA_HOME
```

RULE: After changing PATH or JAVA_HOME, close and reopen the terminal. Existing terminal windows often keep the old environment.

12. Complete comparison

Point	JVM	Runtime environment	JDK
Main purpose	Execute bytecode	Provide execution support	Develop and run software
Contains compiler	No	No	Yes - javac
Contains JVM	It is the JVM	Yes	Yes, as part of runtime support
Contains core libraries	Uses them	Yes	Yes
Used by	Every running Java application	Deployed/running applications	Developers, build systems and tooling
Platform-dependent	Implementation is platform-specific	Distribution is platform-specific	Distribution is platform-specific
Typical beginner install	Not installed alone	Not usually chosen alone today	Install a JDK
Modern note	Specification plus implementations	May be vendor-provided or custom-built	JDK 11+ has no old nested jre/ image

13. Exam, viva and interview answers

30-SECOND ANSWER: JVM is the virtual machine that loads and executes Java bytecode. A Java runtime environment provides the JVM, core libraries and supporting components required to run applications. The JDK is the complete development kit that provides the runtime plus tools such as javac, jar, javadoc and jlink. In modern JDK 11-and-later layouts, the old separate jre/ image is no longer included.

Question	Strong answer
Can JDK run Java programs?	Yes. A JDK contains the launcher, JVM implementation and runtime libraries needed to run programs.
Can JVM compile Java source?	The JVM executes bytecode. javac, a JDK tool, normally compiles source code. The JVM also has JIT compilers that compile bytecode to native code while running.
Is JRE still used as a concept?	Yes, runtime environment remains a useful concept. But the old separate Oracle/OpenJDK JRE download and nested jre/ layout changed; JDK 11+ has no JRE image inside the JDK.
Why is JVM platform-dependent?	It must interact with a specific OS and CPU while implementing common JVM behavior.
What should a beginner install?	A current JDK, preferably an LTS release for stable learning and projects.

14. Practice and revision

- Run java -version and javac -version and write down both outputs.
- Draw the conceptual relationship between JDK, runtime environment and JVM.
- Name five JDK tools and give one use for each.
- Explain why the phrase JDK = JRE + tools is conceptually useful but historically simplified.
- Create ToolDemo.java, compile it, inspect it with javap and package it into a JAR.
- Print java.home and java.vm.name from a program.
- Try jlink with java.base and run the java executable inside the generated folder.

Term	One-line memory statement
JVM	Executes class-file bytecode on the local platform.
Runtime environment	JVM plus libraries and runtime support.
JDK	Runtime plus development, packaging and diagnostic tools.
javac	Source-to-bytecode compiler.
java	Launcher that starts a JVM and application.
jlink	Builds a custom modular runtime image.
JAVA_HOME	Conventional pointer to the selected JDK home.
PATH	Lets the terminal locate java, javac and other tools.

Official sources and further learning

These guides intentionally use official Java and OpenJDK documentation for the technical facts.

Java learning portal: Open official source - Official beginner and advanced Java learning material.

JDK 26 documentation: Open official source - Current Java SE documentation, tools and API references.

Java SE specifications: Open official source - Java Language Specification and Java Virtual Machine Specification.

OpenJDK project: Open official source - Open-source reference implementation and release information.

Oracle Java downloads: Open official source - Current JDK downloads and LTS information.

Oracle migration guide - JDK/JRE layout: Open official source - Explains that JDK 11 and later has no JRE image.

JDK tool specifications: Open official source - Official command documentation for java, javac, jar, jlink and other tools.

VERSION NOTE: Java evolves every six months. The concepts in this guide are stable, while exact tool options and release features may change. For long-term learning, JDK 25 is the current LTS release; JDK 26 is the current feature release as of July 2026.